VINUNIVERSITY

Lecture 11: Objects, Lists and Sequences (Part 1)

Understand sequences, list, and list methods.

Pham Huy Hieu & Kok-Seng Wong | Fall 2025

Oct. 21, 2025

MOTIVATIONAL EXAMPLE 1



[source]





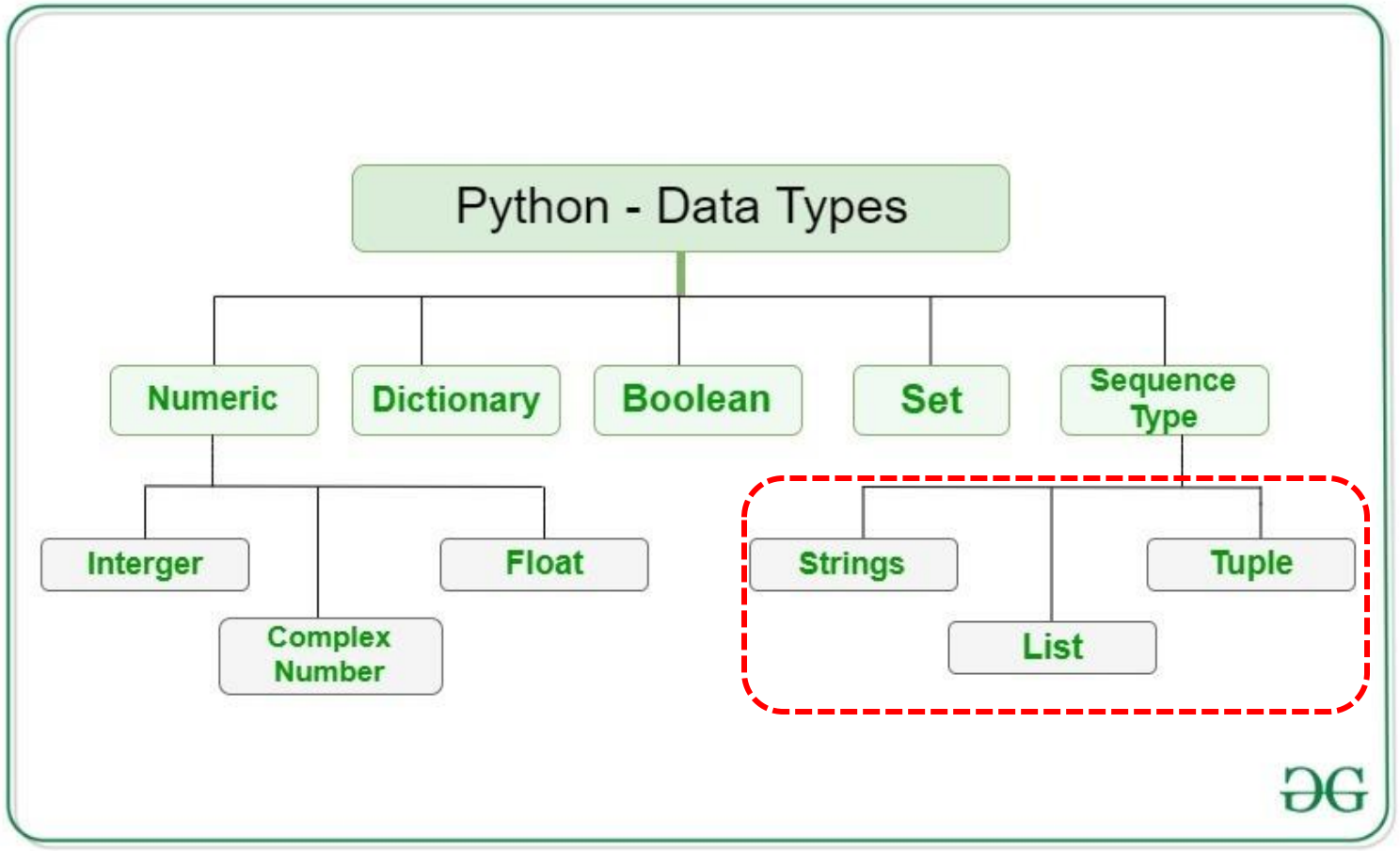
LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- use indexing and slicing to access list elements in Python.
- manipulate lists that hold a collection of objects with the built-in methods.
- understand key operations in lists, e.g. concatenation and repetition.
- understand and explain the differences between strings and lists.

LISTS ARE SEQUENCE TYPES

- Sequence types: program can iterate over them (iterables).



[source]

PYTHON LISTS

- One of the four built-in data structures (together with tuples, dictionaries, and sets) in Python, used to store ordered items.
- It can store different types of items – but not common.
- Use square brackets `[]` and commas (`,`) to separate the elements.

```
my_enemy = ["Thomas", "Billy", "Nam"]
```

Empty List

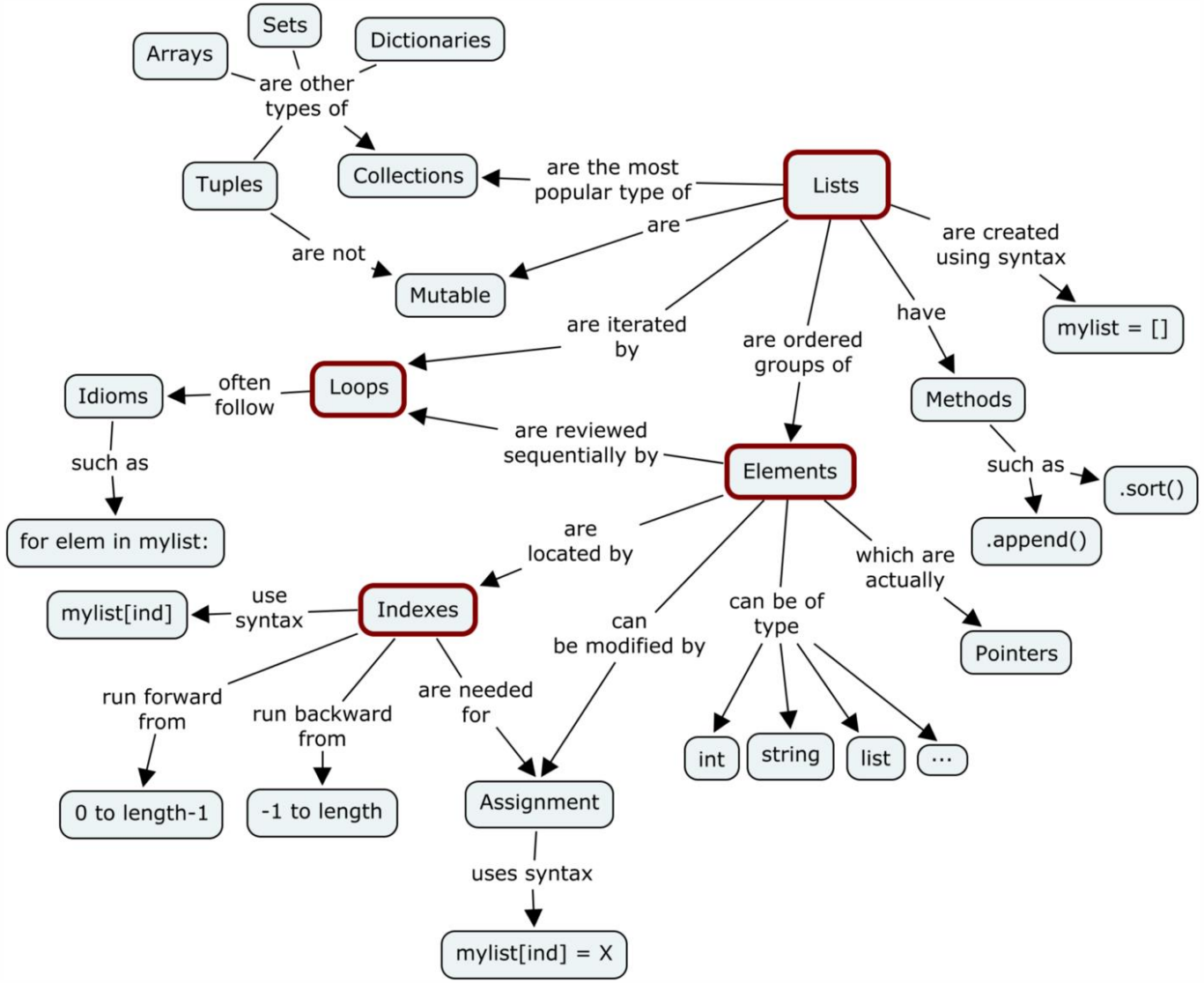
```
my_friend = []
```

LISTS - OUTPUTS

```
a = [13, 10, -1]
print(a)
```

```
my_enemy = ["Thomas", "Billy", "Nam"]
my_friend = []
print(my_enemy, my_friend)
```

PYTHON LISTS - MUTABLE



- The list is a single object that manages an internal array of references to other objects.
- A list itself is one object in memory.
- It has its own memory address, identity in Python.
- It doesn't directly hold all the items inside its structure, instead, it keeps references (pointers) to them.

EXAMPLE 1

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]

print(list1 == list2) # Output: ?
print(list1 is list2) # Output: ?

list1[0] = 100
print(list1)  # Output: ?
print(list2)  # Output: ?
```

LIST OBJECT

```
my_list = [1, 2, [3, 4], "hello"]
```

Here, `my_list` is a single list object, but it contains references to:

- An integer object `1`
- An integer object `2`
- Another list object `[3, 4]`
- A string object `"hello"`

Each of these elements can reside at different memory locations, but the list keeps track of them through references.



EXAMPLE

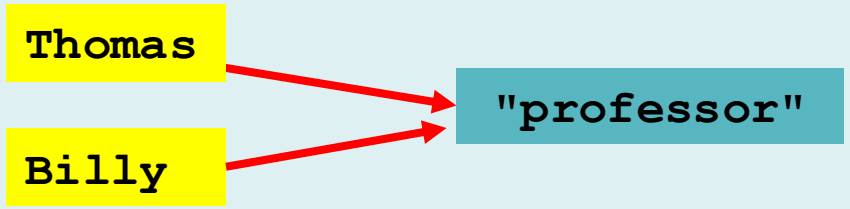
```
my_list = [1, 2, [3, 4], "hello"]
```

```
Original list: [1, 2, [3, 4], 'hello']

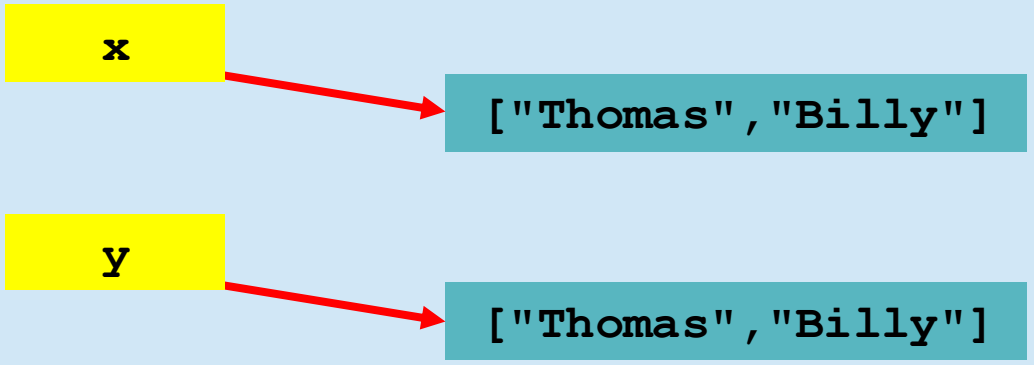
Memory addresses of elements in the list:
Element 0 (1) is stored at memory address: 140496751306032
Element 1 (2) is stored at memory address: 140496751306064
Element 2 ([3, 4]) is stored at memory address: 140496753540352
Element 3 (hello) is stored at memory address: 140496753508912
my_list[2][0] = 99
```

IDENTICAL VS. EQUIVALENT

```
>>> Thomas = "professor"
>>> Billy = "professor"
>>> Thomas is Billy
True
```




```
>>> x = ["Thomas", "Billy"]
>>> y = ["Thomas", "Billy"]
>>> x is y
False
```



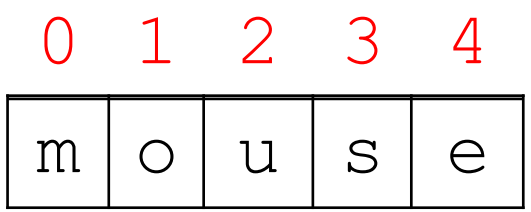
Identical vs. Equivalent

"If two objects are identical, they are also equivalent, but if they are equivalent, they are not necessarily identical."

STRINGS VS LISTS

string

- `s = "mouse"`

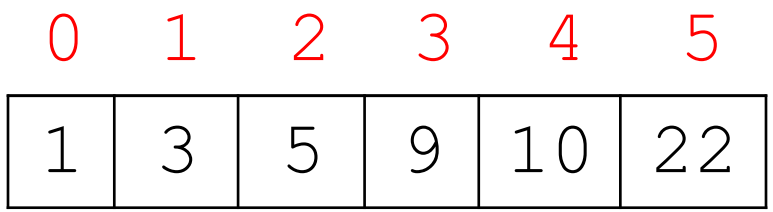


- Put characters in quotes " "
- Access characters with `[]`

```
s[1]="m"
s[6]-> error (why?)
s[1:3]="ou"
len(s)=5
```

list

- `x =[1,3,5,9,10,22]`



- Put elements inside `[]`, separate by commas
- Access elements with `[]`

```
x[3]=19
x[2:5]=[5,19,10]
len(x)=6
```

Mutable

LIST DATA TYPE METHODS



`list.append(x)`

Add an item to the end of the list. Equivalent to `a[len(a):] = [x]`.

`list.extend(iterable)`

Extend the list by appending all the items from the iterable. Equivalent to `a[len`

`list.insert(i, x)`

Insert an item at a given position. The first argument is the index of the element before which to insert, so `a.insert(0, x)` inserts at the front of the list, and `a.insert(len(a), x)` is equivalent to `a.append(x)`.

`list.remove(x)`

Remove the first item from the list whose value is equal to *x*. It raises a `ValueError` if there is no such item.

`list.pop([i])`

Remove the item at the given position in the list, and return it. If no index is specified, `a.pop()` removes and returns the last item in the list. (The square brackets around the *i* in the method signature denote that the parameter is optional, not that you should type square brackets at that position. You will see this notation frequently in the Python Library Reference.)

`list.`**`clear`**`()`

Remove all items from the list. Equivalent to `del a[:]`.

`list.`**`index`**`(`*x*`[,` *start*`[,` *end*`]])`

Return zero-based index in the list of the first item whose value is equal to *x*. Raises a `ValueError` if there is no such item.

The optional arguments *start* and *end* are interpreted as in the slice notation and are used to limit the search to a particular subsequence of the list. The returned index is computed relative to the beginning of the full sequence rather than the *start* argument.

`list.`**`count`**`(`*x*`)`

Return the number of times *x* appears in the list.

`list.`**`sort`**`(*, key=None, reverse=False)`

Sort the items of the list in place (the arguments can be used for sort customization, see `sorted()` for their explanation).

`list.`**`reverse`**`()`

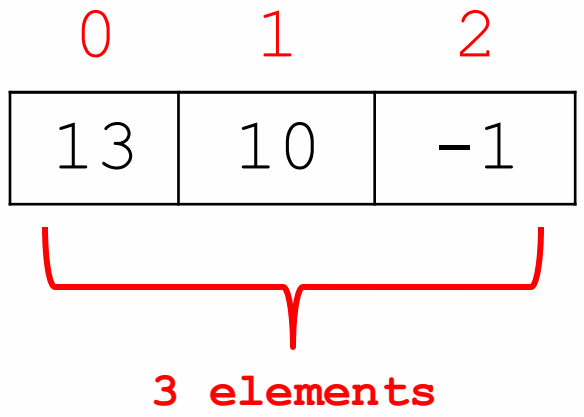
Reverse the elements of the list in place.

`list.`**`copy`**`()`

Return a shallow copy of the list. Equivalent to `a[:]`.

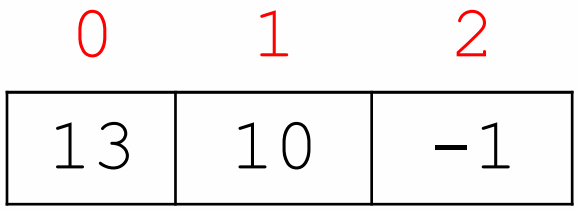
INDICES AND ORDERING

```
>>> a = [13, 10, -1]
>>> a
[13, 10, -1]
>>> len(a)
3
>>> a[0]
13
>>> a[4]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```



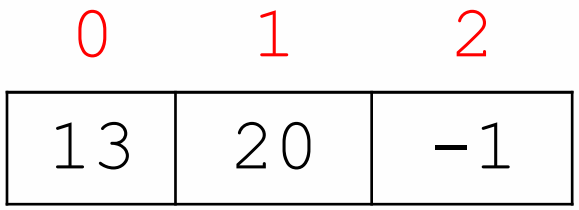
CHANGING ELEMENTS

```
>>> a = [13, 10, -1]
```



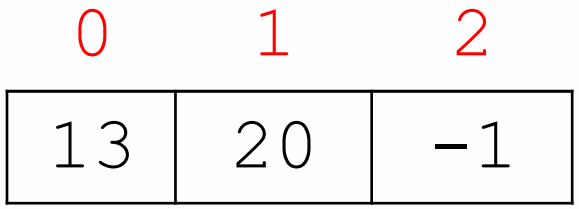
CHANGING ELEMENTS

```
>>> a = [13, 10, -1]
>>> a[1] = 20
```



CHANGING ELEMENTS

```
>>> a = [13, 10, -1]
>>> a[1] = 20
>>> a
[13, 20, -1]
```



CHANGING ELEMENTS

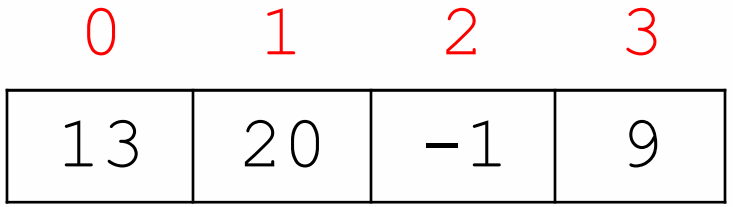
```
>>> a = [13, 10, -1]
>>> a[1] = 20
>>> a
[13, 20, -1]
>>> a.append(9)
```

0	1	2	3
13	20	-1	9

`append()` - appends an element to the end of the list

CHANGING ELEMENTS

```
>>> a = [13, 10, -1]
>>> a[1] = 20
>>> a
[13, 20, -1]
>>> a.append(9)
>>> a
[13, 20, -1, 9]
```



CHANGING ELEMENTS

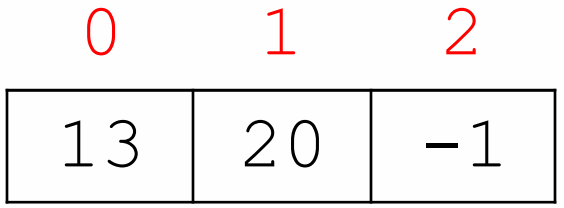
```
>>> a = [13, 10, -1]
>>> a[1] = 20
>>> a
[13, 20, -1]
>>> a.append(9)
>>> a
[13, 20, -1, 9]
>>> a.pop()
9
```

0	1	2
13	20	-1

`pop()` - removes the element at the specified position (default value is -1)

CHANGING ELEMENTS

```
>>> a = [13, 10, -1]
>>> a[1] = 20
>>> a
[13, 20, -1]
>>> a.append(9)
>>> a
[13, 20, -1, 9]
>>> a.pop()
>>> a
[13, 20, -1]
```




```
>>> a.append(9)
>>> a??
```

APPEND VS EXTEND

```
price_A = [2.35, 122, 4.853]
price_B = [134, 0.855]
price_C = [10.01]

price_A.append(price_B)
print(price_A)
```

A list is an object!

```
[2.35, 122, 4.853, [134, 0.855]]
```

```
1, 134, 0.855]
```

CLASS PRACTICE

Sample Code:

```
stud_age = []
age = int(input("Enter your age: "))
if age > 10:
    stud_age.append(age)
else:
    stud_age.pop()

print(stud_age)
```

Output:

How to avoid this runtime error?

CLASS PRACTICE

Sample Code:

```
from random import choice
names = ["Ali", "Baba", "Chris", "David", "Enver"]
print(choice(names))
```

Output:

CLASS PRACTICE

Sample Code:

```
import random

stud_age = [22, 19, 18]
age = random.choice(range(1, 25))

if age > 18:
    stud_age.append(age)
else:
    stud_age.pop()

print(stud_age)
```

Output:



CLASS PRACTICE

Sample Code:

```
import random

lucky_draw = []
res = random.choice(range(1, 20))

if res < 10:
    lucky_draw.append(res)
else:
    lucky_draw.append(sorry)

print(lucky_draw)
```

Output:



CLASS PRACTICE

Sample Code:

```
import random

lucky_draw = []
res = random.choice(range(1, 20))

if res < 10:
    lucky_draw.append(res)
else:
    lucky_draw.pop()

print(lucky_draw)
```

Output:



INSERT ELEMENT(S)

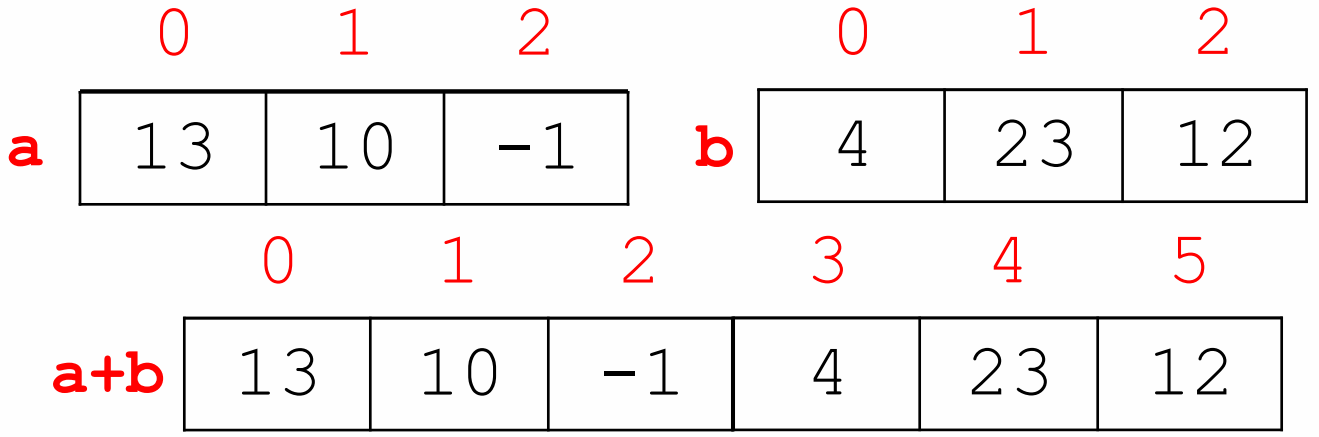
```
>>> a = [13, 10, -1]
>>> a.insert(1,20)
>>> a
[13, 20, 10, -1]
>>> a[1:3] = [2, 8]
>>> a
[13, 2, 8, -1]
```

`insert()`- add an item at the specified position

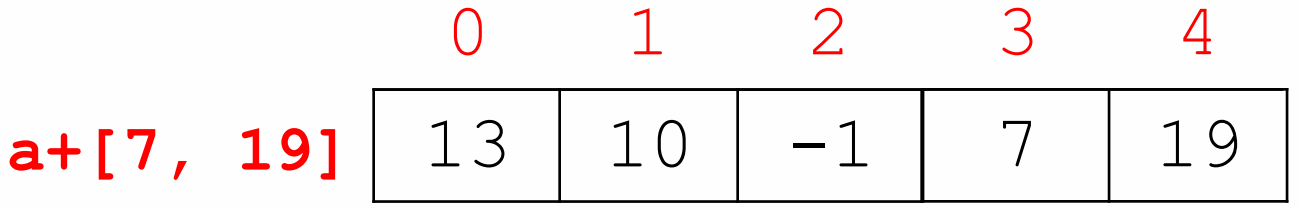
To replace the original elements

LISTS CONCATENATION

```
>>> a = [13, 10, -1]
>>> b = [4, 23, 12]
>>> a + b
[13, 10, -1, 4, 23, 12]
```




```
>>> print(a + [7, 19])
[13, 10, -1, 7, 19]
```




```
>>> print(7 + [7, 19])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'list'
```

LISTS REPETITION

```
>>> list_1 = [1, 3, 5]
>>> list_1 * 4
[1, 3, 5, 1, 3, 5, 1, 3, 5, 1, 3, 5]
>>> print([0] * 3)
[0, 0, 0]
>>> list_1 * 2 + [8,9]
[1, 3, 5, 1, 3, 5, 8, 9]
>>> list_1 * 2 + [8,9] * 3
[1, 3, 5, 1, 3, 5, 8, 9, 8, 9, 8, 9]
>>> (list_1 +[8,9]) * 3
[1, 3, 5, 8, 9, 1, 3, 5, 8, 9, 1, 3, 5, 8, 9]
```

COUNT AND INDEX

```
>>> friends = ["James", "Ali", "Michelle", "Ben"]
>>> friends.count("Ali")
1
>>> friends.index("Ben")
3
>>> friends.index("Baba")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: 'Baba' is not in list
```

TRANSFORM LIST TO STRINGS

- **`list(s)`**: returns a list with every character from `s`
- **`split()`**: splits a string on a character parameter
- **`"".join()`**: turns a list of characters into a string

```
>>> s1 = "Hello"
>>> list(s1)
['H', 'e', 'l', 'l', 'o']
>>> s2 = "I<3U"
>>> s2.split("3")
['I<', 'U']
```

```
>>> s3 = ['H', 'e', 'l', 'l', 'o']
>>> "".join(s3)
'Hello'

>>> s4 = ['ALI', 'BABU']
>>> "*".join(s4)
'ALI*BABU'
```

SORTING ELEMENTS IN A LIST

- **`sorted()`** : create a new list containing a sorted version of the list it is given (can assign the returned data to a **new variable**)
- **`sort()`** : modify the list it is called on

```
>>> fruit_list = ["carrot", "banana", "apple", "eggfruit", "durian"]
>>> sorted(fruit_list)
['apple', 'banana', 'carrot', 'durian', 'eggfruit']
>>> fruit_list
['carrot', 'banana', 'apple', 'eggfruit', 'durian']
>>> new_list = sorted(fruit_list)
>>> new_list
['apple', 'banana', 'carrot', 'durian', 'eggfruit']
>>> fruit_list.sort()
>>> fruit_list
['apple', 'banana', 'carrot', 'durian', 'eggfruit']
```

SORT STRINGS USING SORTED()

```
>>> str = "Ali Baba"
>>> sorted(str)
[' ', 'A', 'B', 'a', 'a', 'b', 'i', 'l']
>>> my_id = "25443234345"
>>> sorted(my_id)
['2', '2', '3', '3', '3', '4', '4', '4', '4', '5', '5']
>>> "".join(my_id)
'25443234345'
```

CLASS PRACTICE

Sample Code:

```
>>> str2 = "Was it a cat I saw?"


>>> sorted_str2 = sorted(str2.split())
```

Output:

```
>>> sorted_str2


>>> " ".join(sorted_str2)
```

REVERSE ELEMENTS IN A LIST

- `reverse()`:

```
>>> fruit_list = ["carrot", "banana", "apple",
                  "eggfruit", "durian"]
>>> fruit_list.reverse()
>>> fruit_list
['durian', 'eggfruit', 'apple', 'banana', 'carrot']
```

- Use `sorted()` with reverse argument

```
>>> sorted(fruit_list, reverse=True)
['eggfruit', 'durian', 'carrot', 'banana', 'apple']
```

More details: https://docs.python.org/3/howto/sorting.html

CLASS PRACTICE

Sample Code:

```
>>> fruit_list = ["carrot", "banana", "apple",
                  "eggfruit", "durian"]
>>> fruit_list.append("tomato")
>>> fruit_list.pop()
>>> fruit_list.pop()
>>> fruit_list.pop()
>>> fruit_list.sort()
>>> fruit_list.pop()
```

```
>>> fruit_list
```

CLASS PRACTICE 1

- **Exercise 1:** Write a function that finds a list's maximum and minimum elements.
- **Exercise 2:** Write a function that removes duplicates from a list.